



PRD — AI Autograder C++ (LLM Grading Engine)



Product Requirements Document (PRD) untuk **AI Grader System** — berfokus pada **LLM Grading Engine** yang mengevaluasi tugas pemrograman C++ secara otomatis.

Versi 1.1 · Status: Draft · Pemilik: Dimas Rizki

1. Ringkasan Eksekutif

AI Grader System adalah platform berbasis web yang melakukan penilaian tugas pemrograman C++ secara otomatis. Sistem menggabungkan **eksekusi kode di sandbox** (kompilasi + test case) dengan **analisis berbasis Large Language Model (LLM)** untuk menghasilkan skor, umpan balik, dan laporan evaluasi yang cepat, objektif, dan konsisten.

Dokumen ini menitikberatkan pada **komponen LLM (AI Engine)**: bagaimana model bahasa besar dipakai untuk menilai kualitas kode, mendeteksi kesalahan, dan menyusun feedback yang terstandarisasi.

2. Latar Belakang & Masalah

Penilaian tugas pemrograman secara manual memiliki beberapa kendala:

- **Beban kerja tinggi** bagi dosen/pengajar, terutama untuk kelas besar.
- **Subjektivitas** — penilaian bisa berbeda antar pengajar maupun dari waktu ke waktu.
- **Feedback lambat**, sehingga siklus pembelajaran mahasiswa terhambat.
- **Inkonsistensi** dalam standar penilaian kualitas kode.

3. Tujuan Proyek

- Membuat proses evaluasi tugas pemrograman lebih **efisien, objektif, dan terukur**.
- **Mengurangi beban penilaian manual** pengajar.
- **Mempercepat pemberian feedback** kepada mahasiswa.
- Meningkatkan kualitas pembelajaran melalui evaluasi yang **detail dan terstandarisasi**.

Tujuan spesifik LLM Engine

- Memberikan penilaian kualitatif (readability, struktur, best practice C++) yang sulit diukur oleh test case saja.
- Menghasilkan umpan balik naratif yang **edukatif** dan **dapat ditindaklanjuti**.
- Menjaga **konsistensi skor** melalui rubrik yang terstruktur.

4. Sasaran Pengguna (Personas)

Pengguna	Kebutuhan Utama	Interaksi dengan LLM Engine
Mahasiswa / Peserta	Mengunggah tugas, melihat skor & feedback cepat	Menerima feedback naratif & saran perbaikan kode
Dosen / Pengajar	Memantau performa, meninjau laporan, mengatur rubrik	Mengonfigurasi bobot rubrik, meninjau & override hasil AI
Admin Sistem	Mengelola model, biaya, dan keamanan	Memilih provider LLM, memantau biaya & rate limit

5. Lingkup Produk

In-Scope

- Unggah soal (PDF) dan source code C++ melalui antarmuka web.
- Eksekusi kode di sandbox Docker (kompilasi + test case).
- Analisis LLM atas kode + hasil eksekusi.
- Skor, feedback, dan laporan evaluasi otomatis.

- Dashboard pemantauan performa untuk pengajar.

Out-of-Scope (v1)

- Penilaian bahasa selain C++ (rencana ekspansi di masa depan).
- Deteksi plagiarisme lintas submisi (backlog).
- Auto-grading untuk proyek multi-file berskala besar / build system kompleks.

6. Alur Kerja Sistem

flowchart TD

```
A["User unggah soal PDF + source code C++"] --> B["Frontend Next.js"]
```

```
B --> C["Backend FastAPI (REST API)"]
```

```
C --> D["PostgreSQL (metadata)"]
```

```
C --> E["Object Storage (R2 / S3)"]
```

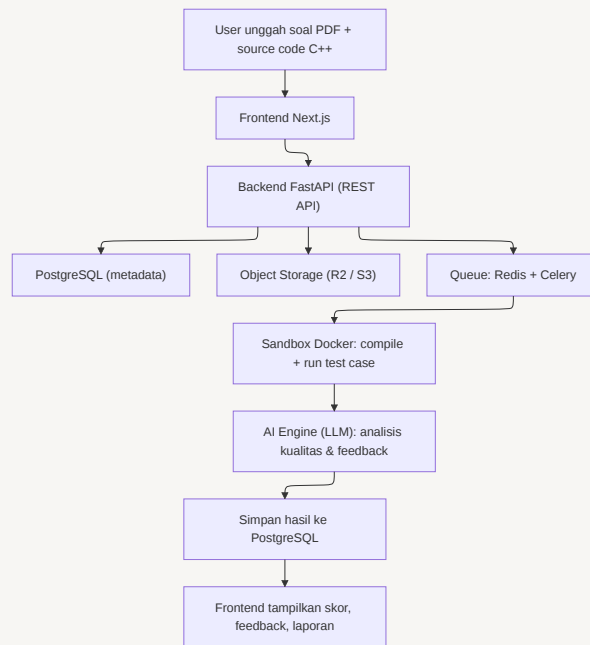
```
C --> F["Queue: Redis + Celery"]
```

```
F --> G["Sandbox Docker: compile + run test case"]
```

```
G --> H["AI Engine (LLM): analisis kualitas & feedback"]
```

```
H --> I["Simpan hasil ke PostgreSQL"]
```

```
I --> J["Frontend tampilkan skor, feedback, laporan"]
```



1. Pengguna mengunggah **soal (PDF)** dan **source code C++** melalui antarmuka web Next.js.
2. File dikirim ke backend FastAPI via REST API.
3. Backend menyimpan metadata ke **PostgreSQL** dan file ke **object storage** (Cloudflare R2 / AWS S3).
4. Pekerjaan penilaian dikirim ke **antrean (Redis + Celery)** untuk pemrosesan asynchronous.
5. Source code dieksekusi di **sandbox Docker**: kompilasi, menjalankan test case, mengumpulkan hasil.
6. Hasil eksekusi + kode dianalisis oleh **AI Engine (LLM)**: OpenAI / Anthropic Claude / Gemini.
7. Hasil evaluasi disimpan ke database dan ditampilkan ke pengguna sebagai **skor, feedback, dan laporan**.

7. Spesifikasi LLM Grading Engine

7.1 Peran LLM

LLM **melengkapi** (bukan menggantikan) hasil eksekusi objektif. Pembagian peran:

- **Sandbox / Test case** → menilai *correctness* (benar/salah output, lolos test).
- **LLM** → menilai *kualitas* (readability, struktur, efisiensi, best practice) dan menyusun *feedback naratif*.

7.2 Input ke Model

- Deskripsi soal (hasil ekstraksi teks dari PDF).
- Source code C++ yang dikirim.
- Hasil kompilasi (sukses/gagal + pesan error).
- Hasil test case (jumlah lolos/gagal, output diff, runtime, memori).
- Rubrik penilaian + bobot tiap kriteria.

7.3 Output Model (Structured)

Model harus mengembalikan **JSON terstruktur**, contoh:

```
{
  "overall_score": 85,
  "rubric_scores": {
    "correctness": 40,
    "code_quality": 20,
    "efficiency": 13,
    "readability": 12
  },
  "feedback_summary": "Solusi sudah benar dan lolos sebagian besar test case...",
  "strengths": ["Penggunaan algoritma dasar tepat", "Penamaan variabel jelas"],
  "improvements": ["Hindari penggunaan variabel global", "Tambahkan edge-case handling"],
  "detected_issues": [
    {
      "type": "performance",
      "severity": "medium",
      "line": 42,
      "message": "Loop  $O(n^2)$  dapat dioptimalkan"
    }
  ],
}
```

```
"confidence": 0.82
```

```
}
```

7.4 Rubrik Penilaian (Contoh Default)

Kriteria	Bobot	Sumber Penilaian
Correctness (test case)	50%	Sandbox (deterministik)
Code Quality / Best Practice	20%	LLM
Efisiensi (kompleksitas/runtime)	15%	LLM + metrik runtime sandbox
Readability & Struktur	15%	LLM

Model pembobotan & perhitungan skor (per sub soal):

- **Keberhasilan test case (deterministik) = 50%** dari nilai tiap sub soal, selalu menjadi komponen utama.
- Jika test case **tidak dapat dijalankan** (gagal kompilasi / tidak tersedia), sub soal tetap mendapat **nilai dasar minimal 25** berdasarkan penilaian LLM atas kode.
- **50% sisanya** dinilai LLM: implementasi algoritma, kesesuaian terhadap soal, serta kualitas & readability kode.
- Pengajar dapat **mengatur bobot tiap sub soal secara manual** melalui kolom input yang muncul setelah ekstraksi soal, sehingga model tidak sekadar menebak bobot — porsi test case tetap 50% per sub soal.

Skor akhir tugas = rata-rata berbobot seluruh sub soal sesuai bobot yang ditetapkan pengajar.

7.5 Prompt Strategy

- **System prompt** mendefinisikan peran sebagai *expert C++ grader* dengan rubrik tetap.
- **Structured output** (JSON schema / function calling) untuk konsistensi parsing.
- **Few-shot examples** untuk kalibrasi skor antar submisi.
- **Temperature rendah** (mis. 0–0.3) demi konsistensi.

- **Guardrail:** model tidak boleh memberi skor correctness yang bertentangan dengan hasil test case deterministik.

7.6 Strategi Multi-Model (Prioritas Gratis)

Prioritas utama: **model gratis tanpa biaya sepeser pun**, memanfaatkan **token gratis harian** bila perlu, dengan hasil semaksimal mungkin.

- **Routing per sub tugas** — tiap bagian penilaian diarahkan ke model yang paling cocok & tersedia:
 - *Analisis & review kode* → diutamakan **Claude** (tier gratis) selama kuota tersedia.
 - *Ringkasan / feedback naratif / penilaian bagian non-kode* → model gratis lain yang tersedia (mis. **Gemini** free tier atau model open-source).
- **Abstraksi provider** sehingga mudah menambah/menukar model gratis tanpa mengubah logika inti.
- **Fallback berjenjang:** bila token gratis harian salah satu model habis atau rate-limited, sistem otomatis beralih ke model gratis berikutnya.
- **Pemantauan kuota harian** per model agar selalu dalam batas gratis.

8. Kebutuhan Fungsional

- ☐ Sistem dapat mengekstrak teks soal dari file PDF.
- ☐ Hasil dari ekstraksi memunculkan kolom pertanyaan untuk bobot penilaian setiap sub soal.
- ☐ Sistem dapat mengompilasi & menjalankan kode C++ di sandbox Docker.
- ☐ Sistem mengirim konteks lengkap (soal, kode, hasil test) ke LLM.
- ☐ LLM mengembalikan skor + feedback dalam format JSON terstruktur.
- ☐ Skor akhir dihitung dari kombinasi hasil sandbox + LLM sesuai bobot rubrik.
- ☐ Pengajar dapat mengatur rubrik & bobot per tugas.
- ☐ Pengajar dapat meninjau dan meng-override hasil AI.

- ☐ Hasil evaluasi ditampilkan **langsung ke mahasiswa** di frontend dalam bentuk tabel / tampilan interaktif (skor per sub soal, feedback deskriptif & poin utama).
- ☐ Feedback harus **konsisten antar submit** (rubrik & kalibrasi yang sama).
- ☐ Sistem merutekan sub tugas ke beberapa **model gratis berbeda** dengan fallback saat kuota habis.
- ☐ Pemrosesan berjalan asynchronous via antrean.

9. Kebutuhan Non-Fungsional

Aspek	Target
Keamanan	Eksekusi kode terisolasi penuh di sandbox Docker (no network, resource limit, timeout)
Performa	Hasil evaluasi tersedia < 2 menit per submit (umum)
Skalabilitas	Mendukung lonjakan submit via worker Celery yang dapat di-scale
Konsistensi	Variasi skor LLM untuk submit identik < 5%
Biaya	Hanya memakai model gratis / token gratis harian — target biaya Rp0; pemantauan kuota harian per model
Reliabilitas	Fallback antar provider LLM; retry pada kegagalan transient

10. Arsitektur Teknologi

Lapisan	Teknologi
Frontend	Next.js
Backend API	FastAPI (Python)
Database	PostgreSQL (Neon / Supabase)
Cache & Queue	Redis
Task Processing	Celery
Sandbox	Docker
File Storage	Cloudflare R2 / AWS S3
AI Engine	OpenAI / Anthropic Claude / Gemini
Deployment Frontend	Vercel

Lapisan	Teknologi
Deployment Backend	Railway / Render

11. Metrik Keberhasilan (KPI)

- **Waktu feedback** turun signifikan dibanding penilaian manual.
- **Korelasi skor AI vs penilaian pengajar** ≥ 0.85 .
- **Tingkat override** pengajar atas skor AI $< 15\%$.
- **Kepuasan mahasiswa** terhadap kualitas feedback (survei).
- **Biaya LLM** tetap Rp0 (dalam kuota gratis / token gratis harian).

12. Risiko & Mitigasi

Risiko	Dampak	Mitigasi
Halusinasi / skor tidak akurat dari LLM	Tinggi	Rubrik ketat, structured output, guardrail terhadap hasil test, review pengajar
Kode berbahaya saat dieksekusi	Tinggi	Sandbox Docker terisolasi, no network, resource & time limit
Kuota token gratis harian habis	Sedang	Routing multi-model gratis + fallback berjenjang, antrean & retry, caching hasil
Ketergantungan satu provider	Sedang	Abstraksi multi-provider + fallback
Inkonsistensi skor antar submisi	Sedang	Temperature rendah, few-shot calibration, rubrik tetap

13. Roadmap (Indikatif)

- ☐ **Fase 1 — MVP:** Unggah, sandbox C++, test case, skor dasar + feedback LLM.
- ☐ **Fase 2 — Rubrik & Dashboard:** Konfigurasi rubrik, dashboard pengajar, override.
- ☐ **Fase 3 — Optimasi:** Multi-provider fallback, kontrol biaya, kalibrasi skor.
- ☐ **Fase 4 — Ekspansi:** Bahasa lain, deteksi plagiarisme, analitik lanjutan.

14. Keputusan Desain

Topik	Keputusan
Tampilan feedback	Ditampilkan langsung ke mahasiswa di frontend (tanpa approval pengajar), dalam bentuk tabel / tampilan interaktif. Feedback berupa teks deskriptif + poin utama, dan harus konsisten antar submisi .
Bobot test case vs LLM	Test case (deterministik) = 50% per sub soal . Bila test case tak bisa dijalankan, nilai dasar minimal 25 . Sisanya dinilai LLM (algoritma, kesesuaian soal, kualitas kode). Bobot sub soal dapat diatur manual oleh pengajar.
Pemilihan model LLM	Prioritas model gratis (token gratis harian). Routing multi-model : Claude untuk analisis kode, model gratis lain (mis. Gemini) untuk bagian lain, dengan fallback antar model gratis.
Privasi source code	Tidak ada batasan khusus — sistem dipakai pribadi oleh pengajar saja. <i>(Catatan: bila nanti dipakai multi-pengguna, kebijakan privasi perlu ditinjau ulang karena kode dikirim ke provider LLM eksternal.)</i>